SET 12

PART A — Short questions and answers

Q1. Compare an entity type and an entity instance using EQUIPMENT as an example.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Compare DROP TABLE and DELETE.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q3. Explain how decomposition removes redundancy from an EQUIPMENT_LAB relation.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Explain the use of the EXISTS operator in a correlated subquery.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Explain the effect of HAVING in: GROUP BY LabID HAVING SUM(Cost) > 10000.

Answer: GROUP BY collects rows with the same value so aggregate functions can calculate a result for each group.

Q6. Describe the purpose of a BEFORE INSERT trigger on the MAINTENANCE table.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q7. Classify MongoDB and Neo4j under the types of NoSQL databases.

Answer: A graph database stores entities as nodes and their connections as relationships.

Q8. Explain the field–value pair structure of a MongoDB document.

Answer: A MongoDB document is one record represented as field–value pairs, similar to a JSON object.

Q9. Describe the output of the MongoDB query: db.equipment.find({Category: "Microscope"}, {EquipmentName: 1, LabID: 1})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Describe how the SET clause updates a node property in Cypher.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: A college maintains a Laboratory Equipment Management System to store laboratory details, equipment

1(a) ER design:

• **LAB**(LabID, LabName, Building); primary key: **LabID**.

• **EQUIPMENT**(EquipmentID, EquipmentName, Category, PurchaseDate, LabID); primary key: **EquipmentID**.

• **MAINTENANCE**(MaintenanceID, EquipmentID, MaintenanceDate, Technician, Cost); primary key: **MaintenanceID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE LAB (
  LabID NUMBER PRIMARY KEY,
  LabName VARCHAR2(100),
  Building VARCHAR2(100)
);
CREATE TABLE EQUIPMENT (
  EquipmentID NUMBER PRIMARY KEY,
  EquipmentName VARCHAR2(100),
```

```
  Category VARCHAR2(100),
  PurchaseDate DATE,
  LabID NUMBER
);
CREATE TABLE MAINTENANCE (
  MaintenanceID NUMBER PRIMARY KEY,
  EquipmentID NUMBER,
  MaintenanceDate DATE,
  Technician VARCHAR2(100),
  Cost NUMBER
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM MAINTENANCE x
JOIN LAB a ON x.LabID = a.LabID
JOIN EQUIPMENT b ON x.EquipmentID = b.EquipmentID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. Graph database (Neo4j/Cypher): Create the required node labels and relationship types exactly as specified in the set. Example pattern:

```
CREATE (a:EntityA {id: 1, name: 'Name A'});
CREATE (b:EntityB {id: 2, name: 'Name B'});
MATCH (a:EntityA {id: 1}), (b:EntityB {id: 2})
CREATE (a)-[:RELATED_TO]->(b);
MATCH (a:EntityA)-[:RELATED_TO]->(b:EntityB)
RETURN a.name, b.name;
```